# Despliegue en AWS

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect

Generado 2026-04-24

# Despliegue en AWS

## Resumen de recursos

| Servicio | Recurso | Propósito |
|---|---|---|
| S3 | 4 buckets (bronze, silver, gold, quarantine) | Data lake |
| KMS | Key + alias | Encriptación SSE-KMS |
| SQS | `ldp-ingest` + DLQ `ldp-ingest-dlq` | Desacoplar S3 eventos del pipeline |
| Lambda | `s3-event-ingest` | S3 PUT → SQS |
| Lambda | `sqs-pipeline-trigger` | SQS → Prefect API |
| Glue | 2 databases, 1 PySpark job | Catalog + normalize heavy |
| RDS | Aurora Postgres Serverless v2 | Source of truth |
| Secrets Manager | `ldp/db/master` | Credenciales DB |
| CloudWatch | Log groups, alarms | Observability |

## Topología

```
        Data producers
             │
             ▼
      ┌──────────────┐
      │  S3 Bronze   │ ◀─── KMS
      └──────────────┘
             │ ObjectCreated
             ▼
      ┌──────────────┐
      │  Lambda      │  s3-event-ingest
      │  (on event)  │
      └──────────────┘
             │ SendMessageBatch
             ▼
      ┌──────────────┐
      │  SQS ingest  │ ◀─── 5 retries → DLQ
      └──────────────┘
             │ EventSourceMapping
             ▼
      ┌──────────────┐
      │  Lambda      │  sqs-pipeline-trigger
      │  (on message)│
      └──────────────┘
             │ HTTP POST /create_flow_run
             ▼
      ┌──────────────┐        ┌──────────────┐
      │  Prefect     │ ◀──▶   │  Worker (ECS)│
      │  Server/Cloud│        │  executes flow│
      └──────────────┘        └──────────────┘
                                     │
                   ┌─────────────────┴─────────────────┐
                   ▼                 ▼                 ▼
            S3 Silver/Gold   Aurora Postgres   Quarantine
```

## Despliegue

### Pre-requisitos

- AWS CLI configurado con credenciales.
- Terraform ≥ 1.5.
- S3 bucket + DynamoDB table para state backend ya creados.

### Primer deploy

```
cd infra/terraform

# Configurar variables
cat > terraform.tfvars <<EOF
environment           = "prod"
s3_bucket_suffix      = "henriquez-20260424"
prefect_api_url       = "https://prefect.example.com/api"
prefect_deployment_id = "xxxx-xxxx-xxxx"
prefect_api_key       = "prf_xxxxxxxx"
vpc_id                = "vpc-xxxxxxxx"
private_subnet_ids    = ["subnet-xxxx", "subnet-yyyy"]
EOF

terraform init
terraform plan -out tfplan
terraform apply tfplan
```

### Outputs relevantes

```
terraform output bronze_bucket          # s3 bucket name
terraform output ingest_queue_url       # SQS URL
terraform output aurora_endpoint        # RDS writer endpoint
terraform output -raw db_secret_arn     # Secrets Manager ARN
```

## Costos estimados (us-east-1, carga moderada)

Asumiendo 10GB/día de Bronze ingest y 1M txns/día:

| Servicio | Mensual estimado |
|---|---|
| S3 Standard (300GB activo) | $7 |
| S3 IA (1TB archivado) | $12 |
| Aurora Serverless v2 (0.5-4 ACU avg) | $150 |
| Lambda invocations (1M/mes) | $2 |
| SQS (10M messages) | $4 |
| KMS keys | $1 |
| CloudWatch logs + metrics | $15 |
| Data transfer | $10 |
| **Total** | **~$200/mes** |

Glue ETL se factura por DPU-hora; 3 workers × 1h/día × 30d × $0.44 = $40/mes adicional si se usa.

## Buenas prácticas aplicadas

### Principio de menor privilegio

Cada Lambda tiene un IAM role con permisos específicos:

```
# Ejemplo: s3-event-ingest solo puede escribir a SQS, no a S3
{
  Effect: "Allow",
  Action: ["sqs:SendMessage", "sqs:SendMessageBatch"],
  Resource: [aws_sqs_queue.ingest.arn]
}
```

### Encriptación en reposo y en tránsito

- S3: SSE-KMS (con key rotation).
- Aurora: encryption at rest con KMS; SSL/TLS para conexiones.
- SQS: `kms_master_key_id` habilita SSE-KMS.
- Secrets Manager: KMS para el secret store.

### Resiliencia

- DLQ en SQS con alarma CloudWatch cuando llegan mensajes.
- `function_response_types = ["ReportBatchItemFailures"]` en EventSourceMapping → sólo re-enqueue los mensajes fallidos, no todo el batch.
- Aurora multi-AZ automático (writer en AZ principal, reader en otra).

### Observabilidad

- Logs de Lambda con retención configurable (default 14 días).
- `enable_continuous_cloudwatch_log = true` en Glue.
- Postgres logs exportados ( `enabled_cloudwatch_logs_exports = ["postgresql"]` ).
- Prometheus remote_write a CloudWatch Metrics (opcional) o a AMP (Amazon Managed Prometheus).

# CI/CD

## Pipeline GitHub Actions (sketch)

```yaml
name: Deploy
on:
  push:
    branches: [main]
    paths: [src/**, infra/**]

jobs:
  test:
    - pip install -e ".[dev]"
    - make test
    - make lint

  terraform-plan:
    needs: test
    - terraform init
    - terraform plan -out tfplan
    - upload tfplan as artifact

  terraform-apply:
    needs: terraform-plan
    environment: production  # requires manual approval
    - terraform apply tfplan

  publish-lambda:
    - zip -r s3_event_ingest.zip src/aws/lambda_handlers/s3_event_ingest.py
    - aws lambda update-function-code ...

  publish-glue:
    - aws s3 cp src/aws/glue_jobs/*.py s3://bronze/glue-scripts/

  register-prefect-deployment:
    - prefect deployment apply ...
```

## Estrategia blue/green para Lambda

Usar alias + versioning; Terraform maneja versions automáticamente cuando cambia el `source_code_hash`.

# Disaster recovery

| Escenario | RTO | RPO | Recuperación |
|---|---|---|---|
| Lambda falla | <1 min | 0 | SQS reintenta automáticamente |
| Aurora down | <5 min | <1s | Failover automático |
| Región down | <1 hora | <15 min | Cross-region backup de S3 + Aurora global |
| Data corruption en Postgres | ~30 min | <24h | PITR restore desde snapshot |
| Bug en pipeline ensució Silver | <2 horas | 0 | Re-procesar Bronze → Silver |

Bronze como **single source of historical truth** es la clave: cualquier bug de transformación se arregla re-ejecutando el flow.

## Runbooks

### Pipeline colgado

1. Verificar Prefect UI por el flow run stuck.
2. Revisar logs del worker (ECS task logs).
3. Verificar SQS queue depth — si alto, el worker no consume.
4. Forzar `FAILED` state en Prefect; el scheduler hará el siguiente run.

### DLQ recibe mensajes

1. CloudWatch alarm `dlq-messages` dispara.
2. Inspeccionar primer mensaje en DLQ con `aws sqs receive-message`.
3. Identificar raíz: payload malformed, Prefect caído, deployment ID inválido.
4. Fix; re-enqueue desde DLQ a main queue.

### Aurora al 100% CPU

1. `SELECT * FROM pg_stat_activity WHERE state = 'active';`
2. Matar queries runaway con `SELECT pg_cancel_backend(pid);`
3. Revisar `pg_stat_statements` top 10 mean_exec_time.
4. Temporal: subir `max_capacity` de Serverless v2.
5. Permanente: optimizar query (índice, rewrite, MView).